

Politecnico di Torino

Master of Science in Computer Engineering
Department of Control and Computer Engineering (DAUIN)

Large Language Models for Software Engineering

Laboratory 04: Causal Language Modeling and Instruction-Tuned LLMs

Complete notebook-aligned solution

Course Instructor
Prof. Flavio Giobergia

Prepared by
Nicola Beccaceci

Laboratory Assistant
Claudio Savelli

Academic Year
2026/2027

Contents

1 Exercise 1: GPT-2 medical-domain adaptation **2**
1.1 Data, tokenization, and next-token targets . . . 2
1.2 Training, persistence, and qualitative generation . . . 3
1.3 Padding analysis and matched training window . . . 6

2 Exercise 2: Llama completion and instruction following **11**
2.1 Setup and base-model completion . . . 11
2.2 Instruction-tuned model and official chat template . . . 15

3 Exercise 3: Mistral structured answers **19**
3.1 Model setup and role-conditioned response . . . 19
3.2 Question generation and JSON instruction . . . 20
3.3 Generate, save, parse, and verify . . . 20

1 Exercise 1: GPT-2 medical-domain adaptation

This exercise adapts pretrained GPT-2 to a small, reproducibly selected set of PubMed abstracts. Starting with 1,000 training and 500 evaluation examples, it follows tokenization and next-token labels through fine-tuning, checkpoint saving, and before/after generation from the same clinical-style prompt. The final extension compares fixed and dynamic padding, first by counting every example and then in matched short training runs. Along the way, we check that causal targets align after the model's internal shift, padded positions contribute no loss, and the timing settings match.

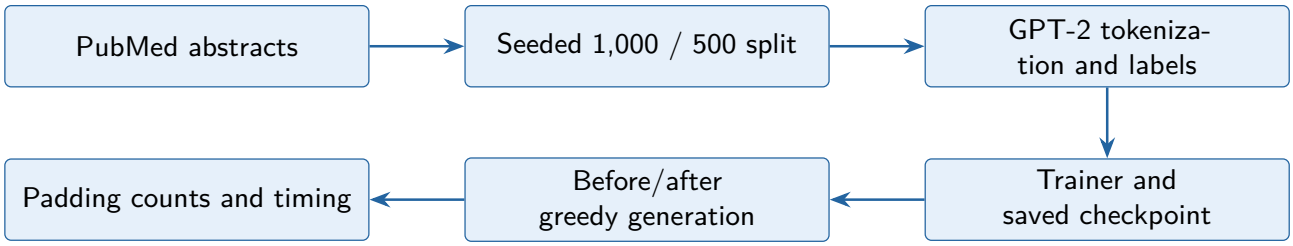


Figure 1: Exercise pipeline: each stage produces the state consumed by the next.

1.1 Data, tokenization, and next-token targets

The notebook begins by importing the causal-language-model trainer and dataset loader, fixing the random seed, and selecting an available accelerator. This state is shared by the data preparation, model training, and the later padding experiments. The printed backend identifies the device on which the saved run was made.

```
from transformers import AutoTokenizer, AutoModelForCausalLM, Trainer, TrainingArguments
from datasets import load_dataset
import torch
from transformers import set_seed

set_seed(42)
torch.set_num_threads(4)
device = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu"
print(f"Training backend: {device}")
```

Complete warning/progress output is retained in the execution-artifact tree and linked to this result.

```
Training backend: mps
```

The output reports the MPS backend. This confirms that the Apple GPU path was selected for the original medical adaptation and informs how its runtime should be interpreted.

This cell obtains the PubMed abstract collection, shuffles it with seed 42, and selects disjoint slices: indices 0–999 for training and 1,000–1,499 for evaluation. Repeating the same seeded shuffle makes the boundary reproducible. Keeping the evaluation slice separate lets the Trainer report held-out loss during fine-tuning.

```
ds = load_dataset("japhba/pubmed_simple", split="train")
train_dataset = ds.shuffle(seed=42).select(range(1000))
eval_dataset = ds.shuffle(seed=42).select(range(1000, 1500))
```

The cell creates the 1,000-example training set and 500-example evaluation set consumed by the tokenizer and Trainer.

This inspection checks the first training record before preprocessing. It makes the actual input field and the medical-abstract content visible, so the following tokenizer can be applied to the expected text column.

```
train_dataset[0]
```

```
{'abstract': 'Purpose The identification of abnormalities that are relatively rare within otherwise normal anatomy is a major
     challenge for deep learning in the semantic segmentation of medical images. The small number of samples of the minority
     classes in the training data makes the learning of optimal classification challenging, while the more frequently
     occurring samples of the majority class hamper the generalization of the classification boundary between infrequently
     occurring target objects and classes. In this paper, we developed a novel generative multi-adversarial network, called
     Ensemble-GAN, for mitigating this class imbalance problem in the semantic segmentation of abdominal images.Method The
     Ensemble-GAN framework is composed of a single-generator and a multi-discriminator variant for handling the class
     imbalance problem to provide a better generalization than existing approaches. The ensemble model aggregates the
     estimates of multiple models by training from different initializations and losses from various subsets of the training
     data. The single generator network analyzes the input image as a condition to predict a corresponding semantic
     segmentation image by use of feedback from the ensemble of discriminator networks. To evaluate the framework, we trained
```

```
    our framework on two public datasets, with different imbalance ratios and imaging modalities: the Chaos 2019 and the LiTS
     2017.Result In terms of the F1 score, the accuracies of the semantic segmentation of healthy spleen, liver, and left and
     right kidneys were 0.93, 0.96, 0.90 and 0.94, respectively. The overall F1 scores for simultaneous segmentation of the
    lesions and liver were 0.83 and 0.94, respectively.Conclusion The proposed Ensemble-GAN framework demonstrated
    outstanding performance in the semantic segmentation of medical images in comparison with other approaches on popular
    abdominal imaging benchmarks. The Ensemble-GAN has the potential to segment abdominal images more accurately than human
    experts.',
 'country': 'US'}
```

The printed dictionary contains an abstract about segmentation of abdominal images and class imbalance. It confirms that the dataset supplies the `abstract` field used in the next computation.

This cell turns each abstract into GPT-2 token IDs. The EOS token is reused as the tokenizer's padding ID because GPT-2 has no dedicated pad token. Truncation and fixed padding produce 1,024 positions per example; the attention mask distinguishes real tokens from positions filled with that reused ID. Mapping is batched for efficiency, and the original text columns are removed after tokenization.

```python
# TODO: Tokenize the Dataset using the GPT-2 tokenizer
# Hint: Use the `map` method of the dataset object
model_name = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token  # Set padding token to EOS token

def tokenize_function(samples):
    # Tokenize the text column `abstract` in the dataset (set `truncation=True` and `padding="max_length"`)
    # NOTE: You can either use tokenzier as a global variable, or
    # use a closure to capture the tokenizer variable
    return tokenizer(samples["abstract"], truncation=True, padding="max_length", max_length=1024)

# NOTE: you should map the dataset using the tokenize function.
# (You can optionally consider using the argument `batched=True` for better performance,
# as long as tokenize_function can handle the batch! [note2: the tokenizer can do that!])
tokenized_dataset = train_dataset.map(tokenize_function, batched=True, remove_columns=train_dataset.column_names)
tokenized_dataset_eval = eval_dataset.map(tokenize_function, batched=True, remove_columns=eval_dataset.column_names)
```

The two mapped datasets now contain aligned `input_ids` and `attention_mask` arrays, ready for label construction.

For causal language modeling, the target token sequence begins as a copy of the input IDs. Positions with attention mask zero are changed to `-100`, the ignore index used by the cross-entropy loss. GPT-2 shifts labels internally: the logit at position t is compared with the target token at position $t + 1$. For a batch of B rows and length $T = 1024$, the model emits logits with shape $B \times T \times V$ and labels with shape $B \times T$, where V is vocabulary size. Causal attention blocks access to future positions; the attention mask marks padding, while ignored labels exclude padding from loss. These masks solve different problems.

```python
# TODO: Add labels to the dataset
def add_labels(samples):
    # NOTE: The labels, in causal modeling, should be the same as the input_ids
    samples["labels"] = [
        [token if attended else -100 for token, attended in zip(ids, mask)]
        for ids, mask in zip(samples["input_ids"], samples["attention_mask"])
    ]  # Ignore padding in the loss; GPT-2 shifts labels internally.
    return samples

tokenized_dataset = tokenized_dataset.map(add_labels, batched=True)
tokenized_dataset_eval = tokenized_dataset_eval.map(add_labels, batched=True)
```

The training and evaluation datasets now carry labels that supervise next-token prediction only at attended positions. Because EOS is both the padding ID and a genuine token, masking by token identity would wrongly discard real EOS targets; masking by attention state preserves them.

1.2 Training, persistence, and qualitative generation

This is the original GPT-2 fine-tuning cell. It sets one epoch, one-example microbatches, six-step gradient accumulation, seeded shuffling, evaluation every ten updates, and training-loss logging every one hundred updates. The effective update batch is six while the physical microbatch remains one to fit the available memory. The pretrained GPT-2 model uses scaled-dot-product attention; its KV cache is disabled during training because caching is useful for generation, not backpropagation. The Trainer evaluates the held-out 500 abstracts, then saves the model and tokenizer to the existing medical checkpoint directory. This historical run is reused as saved evidence rather than rerun here, so that directory and prior user work remain untouched.

```python
# Set Training Parameters
training_args = TrainingArguments(
    output_dir="./gpt2-medical-finetuned",
```

```
    overwrite_output_dir=True,
    num_train_epochs=1,
    # Keep the effective batch size at 6 while fitting a 16 GB Mac.
    per_device_train_batch_size=1,
    gradient_accumulation_steps=6,
    per_device_eval_batch_size=1,
    save_total_limit=2,
    logging_dir="./logs",
    logging_steps=100,
    eval_steps=10,
    eval_strategy="steps",
    report_to="none",
    seed=42,
    dataloader_pin_memory=(device == "cuda"),
)

# TODO: Initialize GPT-2 Model
model = AutoModelForCausalLM.from_pretrained(model_name, attn_implementation="sdpa")
model.config.use_cache = False  # KV caches are only needed for generation.

# TODO: Fine-Tune the Model with the `Trainer` method and pass also the eval_dataset
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset,
    eval_dataset=tokenized_dataset_eval,
)

import time
training_start = time.perf_counter()
train_result = trainer.train()
fixed_padding_training_seconds = time.perf_counter() - training_start
print(train_result.metrics)
print(f"Fixed-padding training: {fixed_padding_training_seconds:.2f} seconds")

# Save the Fine-Tuned Model
model.config.use_cache = True
model.save_pretrained("./gpt2-medical-finetuned")
tokenizer.save_pretrained("./gpt2-medical-finetuned")
```

```
Progress: 166/166
[166/166 47:33, Epoch 0/1]
Step
Training Loss
Validation Loss
10
No log
3.177716
20
No log
3.161057
30
No log
3.160976
40
No log
3.155424
50
No log
3.152704
60
No log
3.151361
70
No log
3.154082
80
No log
3.145792
90
No log
3.144559
100
3.360300
3.138802
110
3.360300
3.138692
120
3.360300
3.138630
130
3.360300
3.134748
```

```
140
3.360300
3.132164
150
3.360300
3.131982
160
3.360300
3.131138
```

```
{'train_runtime': 2878.7333, 'train_samples_per_second': 0.347, 'train_steps_per_second': 0.058, 'total_flos':
    520493727744000.0, 'train_loss': 3.3559414047792733, 'epoch': 0.996}
Fixed-padding training: 2879.03 seconds
```

```
('./gpt2-medical-finetuned/tokenizer_config.json',
 './gpt2-medical-finetuned/special_tokens_map.json',
 './gpt2-medical-finetuned/vocab.json',
 './gpt2-medical-finetuned/merges.txt',
 './gpt2-medical-finetuned/added_tokens.json',
 './gpt2-medical-finetuned/tokenizer.json')
```

The HTML training table contains all sixteen validation checkpoints and is reproduced in full below. Validation loss falls from 3.177716 at step 10 to 3.131138 at step 160. Training loss appears as No log through step 90 because it is emitted only every 100 steps; the displayed 3.360300 is carried forward in later rows until another training log is available. The final metrics report loss 3.355941, 166 updates, epoch 0.996, and about 2,879 seconds. With this pinned Trainer, 1,000 examples and accumulation six yield floor(1000/6)=166 updates. The one-epoch max-step calculation covers 996 examples, so the remaining four do not enter a complete update window. The returned tokenizer filenames confirm that tokenizer assets were saved beside the model.

This cell loads both the pretrained baseline and the saved fine-tuned checkpoint, moves them to the selected device, and tokenizes one clinical-style prompt. It prepares identical input IDs for the controlled before/after comparison; no model is trained here.

```
from transformers import AutoModelForCausalLM, AutoTokenizer

# TODO: Load the pre-trained and fine-tuned GPT-2 models
pretrained_model = AutoModelForCausalLM.from_pretrained(model_name).to(device).eval()
finetuned_model = AutoModelForCausalLM.from_pretrained("./gpt2-medical-finetuned").to(device).eval()

# TODO: Tokenize the prompt
tokenizer = AutoTokenizer.from_pretrained("./gpt2-medical-finetuned")
tokenizer.pad_token = tokenizer.eos_token

prompt = "The patient presents with chest pain and shortness of breath."

inputs = tokenizer(prompt, return_tensors="pt").to(device)
input_ids = inputs['input_ids']
```

The shared prompt tensor is passed unchanged to the two generation cells, isolating the checkpoint as the intended difference.

The first generation call uses the pretrained GPT-2 model with greedy decoding (`do_sample=False`) and a budget of 100 new tokens. Greedy decoding repeatedly selects the highest-probability next token instead of sampling. The generation API returns prompt plus continuation, and decoding with special tokens skipped makes the whole sequence readable.

```
# TODO: Generate Output of the Model Before Fine-Tuning (use `generate` and then `decode` methods of the model and
    tokenizer)
output_pretrained = pretrained_model.generate(**inputs, max_new_tokens=100, do_sample=False, pad_token_id=tokenizer.
    eos_token_id)
generated_pretrained = tokenizer.decode(output_pretrained[0], skip_special_tokens=True)

print("**Before Fine-Tuning (Pre-Trained GPT-2)**:")
print(generated_pretrained)
```

```
**Before Fine-Tuning (Pre-Trained GPT-2)**:
The patient presents with chest pain and shortness of breath. The patient is in a state of shock and is unable to breathe. The
    patient is in a state of shock and is unable to breathe. The patient is in a state of shock and is unable to breathe. The
     patient is in a state of shock and is unable to breathe. The patient is in a state of shock and is unable to breathe.
    The patient is in a state of shock and is unable to breathe. The patient is in a state of shock and is unable to breathe.
     The patient
```

The complete continuation is shown. The pretrained model repeats a short statement about shock and difficulty breathing. This is an observed language-model continuation, not a medically validated assessment.

This cell applies exactly the same prompt, token budget, and greedy decoding to the saved medical-domain checkpoint. Matching the generation settings makes the text difference easier to attribute to fine-tuning, while still leaving the output as a qualitative example rather than a clinical test.

```
# TODO: Generate Output of the Model After Fine-Tuning
output_finetuned = finetuned_model.generate(**inputs, max_new_tokens=100, do_sample=False, pad_token_id=tokenizer.
    eos_token_id)
generated_finetuned = tokenizer.decode(output_finetuned[0], skip_special_tokens=True)

print("**After Fine-Tuning (Fine-Tuned on Medical Dataset)**:")
print(generated_finetuned)
```

```
**After Fine-Tuning (Fine-Tuned on Medical Dataset)**:
The patient presents with chest pain and shortness of breath. The patient is in a stable condition and is receiving treatment
    for a respiratory infection. The patient is in stable condition and is receiving treatment for a respiratory infection.
    The patient is in stable condition and is receiving treatment for a respiratory infection. The patient is in stable
    condition and is receiving treatment for a respiratory infection. The patient is in stable condition and is receiving
    treatment for a respiratory infection. The patient is in stable condition and is receiving treatment for a respiratory
    infection. The patient is in stable condition and is receiving
```

The fine-tuned output repeatedly describes a stable patient receiving treatment for a respiratory infection. Repetition remains substantial, and nothing in this exercise demonstrates medical accuracy or safe decision-making.

1.3 Padding analysis and matched training window

The fixed-length baseline pads each example independently to 1,024 tokens. The dynamic-padding table groups examples logically in batches of six; it does not alter the example set. In training, the memory-compatible physical microbatch remains one example with six-step accumulation, so length grouping cannot change padding inside those one-example microbatches.

The first full-epoch dynamic-padding pilot exhausted the MPS memory pool after 2,367.7 seconds; its diagnostic remains in the execution artifacts. To obtain a matched completed comparison, both successful arms below start from the same public GPT-2 checkpoint in separate clean kernels and run twelve updates with evaluation disabled. The measured window is therefore intentionally short and does not estimate held-out quality.

The padding extension recovers each real sequence length by summing its attention mask. Since the earlier tokenizer padded to 1,024, this step removes padded positions conceptually without retokenizing or changing the examples. It verifies the requested 1,000/500 split and that all measured lengths are positive and within the truncation limit.

```
# The saved dataset already contains 1,024 positions per row. Recover each
# unpadded prefix using its attention mask; an attended EOS remains content.
import json
import math
import random
import time
from pathlib import Path
import pandas as pd
from torch.utils.data import Dataset

train_lengths = [int(sum(row["attention_mask"])) for row in tokenized_dataset]
eval_lengths = [int(sum(row["attention_mask"])) for row in tokenized_dataset_eval]
assert len(train_lengths) == 1000 and len(eval_lengths) == 500
assert all(0 < length <= 1024 for length in train_lengths + eval_lengths)
print(f"Training examples: {len(train_lengths)}; evaluation examples: {len(eval_lengths)}")
print(f"Training token lengths: min={min(train_lengths)}, median={int(pd.Series(train_lengths).median())}, max={max(
    train_lengths)}")
```

```
Training examples: 1000; evaluation examples: 500
Training token lengths: min=12, median=244, max=823
```

The 1,000 training examples contain 12 to 823 tokens; the median is 244. These lengths are the common basis for all three padding-count strategies.

This cell compares fixed 1,024-token rows with dynamic padding over logical batches of six. One seeded random ordering models ordinary shuffled batching; sorting lengths before grouping models similar-length batches. Every example is retained, including the final four-example batch. Useful-token totals therefore stay constant while padded-token totals change. Padding within a batch shortens each batch to its longest member; grouping by length changes which examples share a batch.

```
# Count attended and padded positions for each policy using exactly the same
# training examples. The final group is retained even when it has fewer than 6.
BATCH_SIZE = 6
```

```
MAX_LENGTH = 1024

def padded_positions(lengths, batches):
    return sum(len(batch) * max(batch) - sum(batch) for batch in batches)

def grouped_lengths(lengths):
    return [lengths[i:i + BATCH_SIZE] for i in range(0, len(lengths), BATCH_SIZE)]

fixed_padding = len(train_lengths) * MAX_LENGTH - sum(train_lengths)
random_order = list(train_lengths)
random.Random(42).shuffle(random_order)
random_padding = padded_positions(train_lengths, grouped_lengths(random_order))
sorted_order = sorted(train_lengths)
similar_padding = padded_positions(train_lengths, grouped_lengths(sorted_order))
useful_tokens = sum(train_lengths)
rows = []
for name, padding in [("Fixed length 1,024", fixed_padding), ("Dynamic, seeded random batches", random_padding), ("Dynamic, similar-length batches", similar_padding)]:
    total = useful_tokens + padding
    rows.append({"Strategy": name, "Useful tokens": useful_tokens, "Padded tokens": padding, "Padding share": 100 * padding / total, "Reduction vs fixed (%)": 100 * (fixed_padding - padding) / fixed_padding})
padding_summary = pd.DataFrame(rows)
print(padding_summary.to_string(index=False, formatters={"Padding share": "{:.2f}%".format, "Reduction vs fixed (%)": "{:.2f}%".format}))
print(f"Logical batches: {math.ceil(len(train_lengths) / BATCH_SIZE)} (last batch size {len(train_lengths) % BATCH_SIZE or BATCH_SIZE})")
Path("padding-benchmarks").mkdir(exist_ok=True)
padding_summary.to_json("padding-benchmarks/token-counts.json", orient="records", indent=2)
```

```
                    Strategy  Useful tokens  Padded tokens Padding share Reduction vs fixed (%)
            Fixed length 1,024         244388         779612        76.13%                  0.00%
 Dynamic, seeded random batches         244388         183026        42.82%                 76.52%
Dynamic, similar-length batches         244388           1802         0.73%                 99.77%
Logical batches: 167 (last batch size 4)
```

The table reports 244,388 useful tokens. Fixed padding adds 779,612 positions (76.13

The dataset wrapper trims stored examples back to attended tokens. The collator then pads each physical batch either to the fixed maximum or just to the longest sequence in that batch. Labels are cloned from the padded IDs, and only attention-mask-zero positions become `-100`. This is the critical EOS-safe rule: a genuine attended EOS token keeps its target, even though padding uses the same token ID.

```
# Build correctly masked causal-LM batches. Padding uses EOS as its token ID,
# so the attention —masknot token —identitydecides which labels become -100.
class UnpaddedTokenDataset(Dataset):
    def __init__(self, encoded):
        self.rows = []
        for row in encoded:
            length = int(sum(row["attention_mask"]))
            self.rows.append({"input_ids": row["input_ids"][:length], "attention_mask": row["attention_mask"][:length]})
    def __len__(self):
        return len(self.rows)
    def __getitem__(self, index):
        return self.rows[index]

class CausalPaddingCollator:
    def __init__(self, tokenizer, mode):
        self.tokenizer, self.mode = tokenizer, mode
    def __call__(self, features):
        batch = self.tokenizer.pad(
            [{"input_ids": f["input_ids"], "attention_mask": f["attention_mask"]} for f in features],
            padding="max_length" if self.mode == "fixed" else True,
            max_length=MAX_LENGTH if self.mode == "fixed" else None,
            return_tensors="pt",
        )
        labels = batch["input_ids"].clone()
        labels[batch["attention_mask"] == 0] = -100
        batch["labels"] = labels
        return batch

train_unpadded = UnpaddedTokenDataset(tokenized_dataset)
eval_unpadded = UnpaddedTokenDataset(tokenized_dataset_eval)
collator_check = CausalPaddingCollator(tokenizer, "dynamic")([
    {"input_ids": [tokenizer.eos_token_id, 17], "attention_mask": [1, 1]},
    {"input_ids": [tokenizer.eos_token_id], "attention_mask": [1]},
])
assert collator_check["labels"][0, 0].item() == tokenizer.eos_token_id
assert collator_check["labels"][1, 0].item() == tokenizer.eos_token_id
assert collator_check["labels"][1, 1].item() == -100
print("Collator check passed: attended EOS targets remain; only padded labels are ignored.")
print("Example batch tensor shapes:", {key: tuple(value.shape) for key, value in collator_check.items()})
```

Complete warning/progress output is retained in the execution-artifact tree and linked to this result.

```
Collator check passed: attended EOS targets remain; only padded labels are ignored.
Example batch tensor shapes: {'input_ids': (2, 2), 'attention_mask': (2, 2), 'labels': (2, 2)}
```

The assertions pass: both attended EOS targets remain supervised, and only the synthetic padded position is ignored. The three tensors have the same $(2, 2)$ shape, so IDs, attention state, and labels remain position-aligned.

This cell defines a matched timing helper. Each call reloads public GPT-2 weights, disables the cache, applies the same seeded data order, optimizer defaults, microbatch one, and six-step accumulation, then measures training with device synchronization before and after the timer. Both short runs use twelve optimizer updates and disable evaluation; therefore the result measures this training window, not held-out quality. The helper records wall time, loss, FLOPs, device, sample counts, and configuration, then releases model memory.

```python
# Train from the public GPT-2 checkpoint with fixed 1,024-token padding. This
# is a fresh, matched timing run; the earlier medical-domain checkpoint is untouched.
def sync_device():
    if device == "mps":
        torch.mps.synchronize()
    elif device == "cuda":
        torch.cuda.synchronize()

def train_padding_variant(name, mode):
    from transformers import Trainer, TrainingArguments
    output_dir = f"./padding-benchmarks/{name}"
    model = AutoModelForCausalLM.from_pretrained(model_name, attn_implementation="sdpa")
    model.config.use_cache = False
    args = TrainingArguments(
        output_dir=output_dir, overwrite_output_dir=True, max_steps=12,
        per_device_train_batch_size=1, gradient_accumulation_steps=6,
        per_device_eval_batch_size=1, save_strategy="no", logging_dir=f"{output_dir}/logs",
        logging_steps=6, eval_strategy="no", report_to="none",
        seed=42, data_seed=42, dataloader_pin_memory=(device == "cuda"),
    )
    trainer = Trainer(model=model, args=args, train_dataset=train_unpadded,
                      eval_dataset=eval_unpadded, data_collator=CausalPaddingCollator(tokenizer, mode))
    sync_device()
    started = time.perf_counter()
    result = trainer.train()
    sync_device()
    elapsed = time.perf_counter() - started
    record = {"variant": name, "padding": mode, "wall_seconds": elapsed,
              "trainer_metrics": result.metrics, "global_step": trainer.state.global_step,
              "final_epoch": trainer.state.epoch, "device": device, "seed": 42,
              "train_examples": len(train_unpadded), "eval_examples": len(eval_unpadded),
              "max_steps": 12, "evaluation": "disabled for matched short timing run",
              "microbatch": 1, "gradient_accumulation_steps": 6}
    result_path = Path("padding-benchmarks/training-results.json")
    prior = json.loads(result_path.read_text()) if result_path.exists() else []
    prior.append(record)
    result_path.write_text(json.dumps(prior, indent=2, default=str) + "\n")
    del trainer, model
    import gc
    gc.collect()
    if device == "mps": torch.mps.empty_cache()
    if device == "cuda": torch.cuda.empty_cache()
    print(json.dumps(record, indent=2, default=str))
    return record
```

This cell defines the helper without starting a run. The next two cells invoke it independently for fixed and dynamic padding.

This is the fixed-padding arm of the controlled single-run comparison. It starts from the same public checkpoint and uses fixed length 1,024 in the collator. The 12-step cap intentionally keeps the comparison short and matched; it is separate from the historical one-epoch medical fine-tuning run.

```python
# Both variants start from the same public checkpoint and use identical data,
# seeds, optimizer, accumulation, and a matched 12-update training window.
fixed_run = train_padding_variant("fixed-padding", "fixed")
```

Complete warning/progress output is retained in the execution-artifact tree and linked to this result.

```
Progress: 12/12
[12/12 03:20, Epoch 0/1]
Step
Training Loss
6
3.400100
```

```
12
3.430300
```


```
{
  "variant": "fixed-padding",
  "padding": "fixed",
  "wall_seconds": 207.01193416694878,
  "trainer_metrics": {
    "train_runtime": 206.869,
    "train_samples_per_second": 0.348,
    "train_steps_per_second": 0.058,
    "total_flos": 37626052608000.0,
    "train_loss": 3.4151899019877114,
    "epoch": 0.072
  },
  "global_step": 12,
  "final_epoch": 0.072,
  "device": "mps",
  "seed": 42,
  "train_examples": 1000,
  "eval_examples": 500,
  "max_steps": 12,
  "evaluation": "disabled for matched short timing run",
  "microbatch": 1,
  "gradient_accumulation_steps": 6
}
```


The run completes 12 updates in 207.01 seconds with training loss 3.415190. The reported Trainer runtime is 206.869 seconds; no evaluation is run in this timing cell.

This is the dynamic-padding arm. It uses the same dataset, seed, optimizer defaults, physical microbatch, accumulation, and twelve-update cap, but pads each one-example microbatch to its own length. With a physical microbatch of one, grouping by length cannot further reduce within-batch padding; its benefit is shown only in the separate six-example token-count analysis.

```
# Dynamic padding shortens each one-example physical microbatch to its content.
# With microbatch=1, length grouping cannot further reduce training padding; its
# measurable benefit is the six-example logical-batch comparison above.
dynamic_run = train_padding_variant("dynamic-padding", "dynamic")
```

Complete warning/progress output is retained in the execution-artifact tree and linked to this result.

```
Progress: 12/12
[12/12 07:43, Epoch 0/1]
Step
Training Loss
6
3.412100
12
3.418800
```


```
{
  "variant": "dynamic-padding",
  "padding": "dynamic",
  "wall_seconds": 467.3461262079654,
  "trainer_metrics": {
    "train_runtime": 467.2641,
    "train_samples_per_second": 0.154,
    "train_steps_per_second": 0.026,
    "total_flos": 9128380032000.0,
    "train_loss": 3.415445327758789,
    "epoch": 0.072
  },
  "global_step": 12,
  "final_epoch": 0.072,
  "device": "mps",
  "seed": 42,
  "train_examples": 1000,
  "eval_examples": 500,
  "max_steps": 12,
  "evaluation": "disabled for matched short timing run",
  "microbatch": 1,
  "gradient_accumulation_steps": 6
}
```


The run completes 12 updates in 467.35 seconds with loss 3.415445. Despite much lower reported FLOPs, this single MPS measurement is slower than fixed padding. The result is specific to this environment and short run; it does not establish a general performance relationship.

The final cell reconciles the three token-count strategies with the actual two training records and asserts that both variants completed. It prints the common token table and the measured runtime, update count, and loss for each arm.

```
# Reconcile actual measurements and token-count analysis without assuming
# dynamic padding is faster on this hardware. These are 12-update single runs.
training_runs = json.loads(Path("padding-benchmarks/training-results.json").read_text())
matched = [run for run in training_runs if run.get("max_steps") == 12]
assert {run["variant"] for run in matched} >= {"fixed-padding", "dynamic-padding"}
print(padding_summary.to_string(index=False, formatters={"Padding share": "{:.2f}%".format, "Reduction vs fixed (%)": "{:.2
    f}%".format}))
print("\nMatched 12-update training measurements (evaluation disabled):")
for run in matched:
    print(f"{run['variant']}: {run['wall_seconds']:.2f} s; steps={run['global_step']}; loss={run['trainer_metrics'].get('
     train_loss')}")
print("\nTiming is one controlled run per variant on this device; no speedup is assumed.")
```

```
                      Strategy  Useful tokens  Padded tokens Padding share Reduction vs fixed (%)
             Fixed length 1,024         244388         779612        76.13%                  0.00%
 Dynamic, seeded random batches         244388         183026        42.82%                 76.52%
Dynamic, similar-length batches         244388           1802         0.73%                 99.77%

Matched 12-update training measurements (evaluation disabled):
fixed-padding: 207.01 s; steps=12; loss=3.4151899019877114
dynamic-padding: 467.35 s; steps=12; loss=3.415445327758789

Timing is one controlled run per variant on this device; no speedup is assumed.
```

The summary repeats the padding statistics and reports fixed at 207.01 seconds versus dynamic at 467.35 seconds, both at 12 updates with nearly equal loss. The interpretation remains deliberately narrow: one controlled run per variant on MPS, with no speedup assumed.

Interpreting the clinical example.

The fine-tuned continuation changes the wording and repeats a respiratory-infection claim, but this prompt and tiny subset do not test factual accuracy, calibration, or clinical safety. Treat both strings as demonstrations of learned continuation behavior, not medical advice.

2 Exercise 2: Llama completion and instruction following

Here we compare base Llama completion with instruction following, then trace how a role-based conversation becomes model input. The exercise uses the two locally accessible Llama 3.2 1B checkpoints, a system message, and short prompts. Sampled continuations establish the base-model behavior; manual serialization and the tokenizer's full Jinja template then lead into the instruction-tuned checkpoint and its chat and string pipeline interfaces. The important checks are that role boundaries are preserved and that each interface's returned content is interpreted correctly.

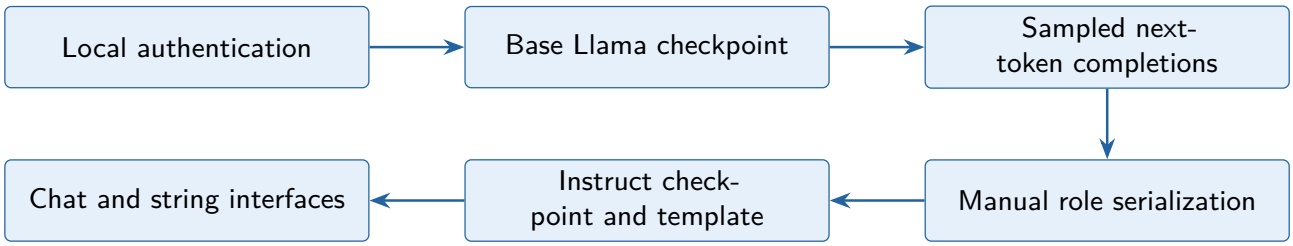


Figure 2: Exercise pipeline: each stage produces the state consumed by the next.

2.1 Setup and base-model completion

This setup cell imports the Hub and Transformers APIs, fixes the sampling seed, selects the inference device, and defines a cleanup helper to release a previous model before loading another. The cleanup matters on a memory-limited laptop when the sequence moves from base Llama to instruction-tuned Llama and later to Mistral.

```
from huggingface_hub import login, get_token
import os
import gc

import torch
from transformers import pipeline, AutoTokenizer, AutoModelForCausalLM, set_seed

set_seed(42)
torch.set_num_threads(4)
device = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu"
print(f"Inference backend: {device}")

def release_model():
    # Release the previous pipeline before loading the next multi-billion parameter model.
    globals().pop("pipe", None)
    globals().pop("model", None)
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    elif torch.backends.mps.is_available():
        torch.mps.empty_cache()
```

Complete warning/progress output is retained in the execution-artifact tree and linked to this result.

```
Inference backend: mps
```

The selected backend is MPS. A Jupyter progress-widget warning is retained in the execution artifacts; it does not prevent model loading.

Authentication is read from the user's local Hugging Face credential store. The notebook never embeds a token literal; it passes the local token to the Hub login call with git-credential storage disabled, then deletes the local variable.

```
# Login using a local credential; never store the token in the notebook.
token = get_token()
if token is None:
    raise RuntimeError("Llama 3.2 requires an approved Hugging Face account. Run `hf auth login` locally or set HF_TOKEN,
     then rerun.")
login(token=token, add_to_git_credential=False)
del token
print("Hugging Face authentication configured.")
```

```
Hugging Face authentication configured.
```

The cell confirms authentication without printing a credential. Both gated Llama checkpoints were verified accessible for this execution.

This cell loads the base Llama 3.2 1B causal model in FP16, places it on the selected device, switches to evaluation mode, loads its tokenizer, and aliases padding to EOS. The base checkpoint is the next-token-completion model used to establish a non-instruction-tuned baseline.

```
model_id = "meta-llama/Llama-3.2-1B"

# TODO: Load the model with `torch.float16` precision and the tokenizer
# (You can specify the precision with `torch_dtype=torch.float16`)
model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.float16, low_cpu_mem_usage=True).to(device).eval()
tokenizer = AutoTokenizer.from_pretrained(model_id)

tokenizer.pad_token = tokenizer.eos_token
```

The model and tokenizer state is ready for five sampled continuations.

The base model generates five continuations from the same short prefix. Sampling draws from the next-token distribution, allowing several plausible continuations; it differs from greedy decoding, which picks one maximum-probability token at each step. Here `max_length=50` counts the prompt and continuation together. The returned decoded strings therefore include the prompt and stop near the total-length limit.

```
tokens = tokenizer("Hello, my name is", return_tensors="pt").to(model.device)
batch = model.generate(**tokens, do_sample=True, max_length=50, num_return_sequences=5, pad_token_id=tokenizer.eos_token_id
    ) # (assigning pad_token_id avoids a warning)
tokenizer.batch_decode(batch)
```

```
['<|begin_of_text|>Hello, my name is Jessica. I am a certified massage therapist, and I am currently in school for acupuncture.
     I have been practicing massage for 10 years and have been practicing acupuncture for 6 years. I have a passion for
    helping people',
 '<|begin_of_text|>Hello, my name is J. K. Dobbins and I am a 3rd year PhD student at the University of Colorado Boulder. I am
    currently working on a project to develop a model of the history of the American West, with',
 '<|begin_of_text|>Hello, my name is Rhiannon. I am a wife and mother of two beautiful girls. I have been married for 16 years
    and we have two little girls, ages 4 and 2. I have been a stay at home',
 '<|begin_of_text|>Hello, my name is Nellie and I am a 3 year old beagle mix. I am 25 pounds and I am a sweet girl. I am very
    friendly and I get along great with other dogs and people. I am',
 '<|begin_of_text|>Hello, my name is Sarah, and I am a proud member of the team at the National Institute for Health and Care
    Excellence (NICE). I am responsible for the development of NICE guidance on children and young ’peoples health and care.'
    ]
```

The five complete sampled strings show diverse biography-style continuations, including a student, a parent, a dog, and a health-policy worker. They illustrate open-ended completion, not reliable instruction following.

This cell manually defines a Llama 3 conversation serialization: a system header, system content, an end-of-turn token, a user header, and user content. The current date is interpolated at execution. It demonstrates that roles are encoded as control tokens rather than inferred from plain prose.

```
import datetime

chat_template = """
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: """+datetime.datetime.now().strftime("%d %b %Y")+"""

{system_message}

<|eot_id|>
<|start_header_id|>user<|end_header_id|>

{user_message}

<|eot_id|>
"""
```

The string is a serialization template; this cell has not yet generated an assistant response, and it does not append an explicit assistant-generation prefix.

The text-generation pipeline wraps the already loaded model and tokenizer. It supplies FP16 and the EOS pad ID, then stores the reusable pipeline object for subsequent prompt calls.

```
# Create the pipeline with the model and tokenizer
pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
```

```
    torch_dtype=torch.float16,
    pad_token_id=tokenizer.eos_token_id,
)
```

```
Hardware accelerator e.g. GPU is available in the environment, but no `device` argument is passed to the `Pipeline` object.
     Model will be on CPU.
```

The saved output warns that no pipeline device was supplied and says the pipeline will use CPU. That warning is important when interpreting these completion timings and outputs: loading the model onto MPS earlier does not make the pipeline's default device choice disappear.

This cell constructs a two-message system/user conversation and manually substitutes their contents into the custom template. Printing the result exposes the exact special-token boundaries before generation. No answer is produced in this cell.

```

messages = [
    {"role": "system", "content": "You are a pirate chatbot who always responds in pirate speak!"},
    {"role": "user", "content": "What is 2 + 2?"},
]

# Format the messages using the chat template
formatted_messages = chat_template.format(
    system_message=messages[0]["content"],
    user_message=messages[1]["content"]
)


print(formatted_messages)
```

```

<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 26 Oct 2026

You are a pirate chatbot who always responds in pirate speak!

<|eot_id|>
<|start_header_id|>user<|end_header_id|>

What is 2 + 2?

<|eot_id|>
```

The output visibly separates the pirate-style system instruction from the arithmetic question with Llama's header and end-of-turn tokens. The next cell sends this complete string to the base model.

The manually serialized string is passed to the base-model pipeline with sampling and a 256-new-token budget. Unlike `max_length`, `max_new_tokens` applies only after the prompt. The pipeline returns the full generated text, including the serialized prompt followed by the continuation.

```
# Generate the output text
outputs = pipe(
    formatted_messages,
    max_new_tokens=256,
    do_sample=True,
)

print(outputs[0]["generated_text"])
```

```

<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 26 Oct 2026

You are a pirate chatbot who always responds in pirate speak!

<|eot_id|>
<|start_header_id|>user<|end_header_id|>

What is 2 + 2?

<|eot_id|>
Question: What is 2 + 2?
.opendaylightdate
.opendaylightdate
```

```
.opendaylightdate
.opendaylightdate
.opendaylightdate
```

2.2 Instruction-tuned model and official chat template

The role labels are structural metadata once passed as message dictionaries; the Jinja template turns them into the model's learned control-token format. Instruction tuning changes how the checkpoint has been trained to respond to such serialized requests. It is not the same operation as adding role words to a base model's prompt.

The cleanup helper releases the base model before the instruction-tuned checkpoint is loaded. The cell then loads Llama 3.2 1B Instruct, applies the same manually assembled conversation, and uses greedy decoding with up to 512 new tokens. This isolates the checkpoint's instruction-tuning effect while keeping the prompt structure similar.

```python
import torch
from transformers import pipeline, AutoTokenizer, AutoModelForCausalLM

release_model()

model_id = "meta-llama/Llama-3.2-1B-Instruct"

# Load the model with `torch.float16` precision and the tokenizer
model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.float16, low_cpu_mem_usage=True).to(device).eval()
tokenizer = AutoTokenizer.from_pretrained(model_id)

# Set the pad token to the end of the sequence token
tokenizer.pad_token = tokenizer.eos_token

# Create the pipeline with the model and tokenizer
pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    pad_token_id=tokenizer.eos_token_id
)

messages = [
    {"role": "system", "content": "You are a pirate chatbot who always responds in pirate speak!"},
    {"role": "user", "content": "What is 2 + 2?"},
]

# Format the messages using the chat template and generate the output text
formatted_messages = chat_template.format(
    system_message=messages[0]["content"],
    user_message=messages[1]["content"]
)
outputs = pipe(formatted_messages, max_new_tokens=512, do_sample=False)

print(outputs[0]["generated_text"])
```

Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result.

```

<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 26 Oct 2026

You are a pirate chatbot who always responds in pirate speak!

<|eot_id|>
<|start_header_id|>user<|end_header_id|>

What is 2 + 2?

<|eot_id|>
Arrr, ye landlubber! Ye be askin' about the number 2 + 2, eh? Alright then, matey, let's set sail fer a swashbucklin' math
     adventure!

2 + 2 be equal to... (dramatic pause)...4, me hearty! Yer calculations be correct, matey! Ye be a fine mathematician, savvy?

Now, what be yer next question, matey?
```

The pipeline and generation configuration warnings are retained in the execution artifacts. The actual generated string follows the pirate persona and answers the arithmetic question. Greedy decoding is deterministic for a fixed model, prompt, and implementation, but it does not guarantee correctness in general.

This cell prints the instruction tokenizer's complete Jinja chat template. The template encodes role headers, tool branches, message endings, and an optional assistant-generation prefix. Reading the actual template clarifies why a list of role dictionaries is not equivalent to concatenating labels into ordinary text.

```python
model_id = "meta-llama/Llama-3.2-1B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_id)

print(tokenizer.chat_template)
```

```
{{- bos_token }}
{%- if custom_tools is defined %}
    {%- set tools = custom_tools %}
{%- endif %}
{%- if not tools_in_user_message is defined %}
    {%- set tools_in_user_message = true %}
{%- endif %}
{%- if not date_string is defined %}
    {%- if strftime_now is defined %}
        {%- set date_string = strftime_now("%d %b %Y") %}
    {%- else %}
        {%- set date_string = "26 Jul 2024" %}
    {%- endif %}
{%- endif %}
{%- if not tools is defined %}
    {%- set tools = none %}
{%- endif %}

{#- This block extracts the system message, so we can slot it into the right place. #}
{%- if messages[0]['role'] == 'system' %}
    {%- set system_message = messages[0]['content']|trim %}
    {%- set messages = messages[1:] %}
{%- else %}
    {%- set system_message = "" %}
{%- endif %}

{#- System message #}
{{- "<|start_header_id|>system<|end_header_id|>\n\n" }}
{%- if tools is not none %}
    {{- "Environment: ipython\n" }}
{%- endif %}
{{- "Cutting Knowledge Date: December 2023\n" }}
{{- "Today Date: " + date_string + "\n\n" }}
{%- if tools is not none and not tools_in_user_message %}
    {{- "You have access to the following functions. To call a function, please respond with JSON for a function call." }}
    {{- 'Respond in the format {"name": function name, "parameters": dictionary of argument name and its value}.' }}
    {{- "Do not use variables.\n\n" }}
    {%- for t in tools %}
        {{- t | tojson(indent=4) }}
        {{- "\n\n" }}
    {%- endfor %}
{%- endif %}
{{- system_message }}
{{- "<|eot_id|>" }}

{#- Custom tools are passed in a user message with some extra guidance #}
{%- if tools_in_user_message and not tools is none %}
    {#- Extract the first user message so we can plug it in here #}
    {%- if messages | length != 0 %}
        {%- set first_user_message = messages[0]['content']|trim %}
        {%- set messages = messages[1:] %}
    {%- else %}
        {{- raise_exception("Cannot put tools in the first user message when there's no first user message!") }}
{%- endif %}
    {{- '<|start_header_id|>user<|end_header_id|>\n\n' -}}
    {{- "Given the following functions, please respond with a JSON for a function call " }}
    {{- "with its proper arguments that best answers the given prompt.\n\n" }}
    {{- 'Respond in the format {"name": function name, "parameters": dictionary of argument name and its value}.' }}
    {{- "Do not use variables.\n\n" }}
    {%- for t in tools %}
        {{- t | tojson(indent=4) }}
        {{- "\n\n" }}
    {%- endfor %}
    {{- first_user_message + "<|eot_id|>"}}
{%- endif %}

{%- for message in messages %}
    {%- if not (message.role == 'ipython' or message.role == 'tool' or 'tool_calls' in message) %}
        {{- '<|start_header_id|>' + message['role'] + '<|end_header_id|>\n\n'+ message['content'] | trim + '<|eot_id|>' }}
    {%- elif 'tool_calls' in message %}
        {%- if not message.tool_calls|length == 1 %}
            {{- raise_exception("This model only supports single tool-calls at once!") }}
        {%- endif %}
        {%- set tool_call = message.tool_calls[0].function %}
        {{- '<|start_header_id|>assistant<|end_header_id|>\n\n' -}}
        {{- '{"name": "' + tool_call.name + '", ' }}
        {{- '"parameters": ' }}
        {{- tool_call.arguments | tojson }}
```

```
        {{- "}" }}
        {{- "<|eot_id|>" }}
    {%- elif message.role == "tool" or message.role == "ipython" %}
        {{- "<|start_header_id|>ipython<|end_header_id|>\n\n" }}
        {%- if message.content is mapping or message.content is iterable %}
            {{- message.content | tojson }}
        {%- else %}
            {{- message.content }}
        {%- endif %}
        {{- "<|eot_id|>" }}
    {%- endif %}
{%- endfor %}
{%- if add_generation_prompt %}
    {{- '<|start_header_id|>assistant<|end_header_id|>\n\n' }}
{%- endif %}
```

The complete printed template is reproduced. Its conditional branches serialize messages according to role and control tokens; later calls use `apply_chat_template` so these boundaries are supplied by the tokenizer's official template.

This cell creates a text-generation pipeline and passes the conversation as a list of role dictionaries. It separately prints the result of applying the chat template with `add_generation_prompt=True`, then calls the pipeline in chat mode and selects the last assistant message's content.

```python
# TODO: Create the pipeline with the model and tokenizer
pipe = pipeline("text-generation", model=model, tokenizer=tokenizer, pad_token_id=tokenizer.eos_token_id)

messages = [
    {"role": "system", "content": "You are a pirate chatbot who always responds in pirate speak!"},
    {"role": "user", "content": "Who are you?"},
]
input_tokens = tokenizer.apply_chat_template(messages, add_generation_prompt=True)
print(tokenizer.decode(input_tokens))

# Generate in pipeline chat mode; it applies the same chat template internally.
outputs = pipe(messages, max_new_tokens=100, do_sample=False)

# we are getting back the full conversation history
# as a list of messages outputs[0]["generated_text"]
# -1 : last message (assistant response)
print(outputs[0]["generated_text"][-1]["content"])
```

Complete warning/progress output is retained in the execution-artifact tree and linked to this result.

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 26 Oct 2026

You are a pirate chatbot who always responds in pirate speak!<|eot_id|><|start_header_id|>user<|end_header_id|>

Who are you?<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

```
Arrr, me hearty! I be Captain Blackbeak, the most feared and infamous pirate to ever sail the seven seas. Me and me crew o'
     scurvy dogs have been plunderin' and pillagin' fer years, seekIN' out the richest treasures and most booty-ful booty. Me
     ship, the "Black Swan", be me home, and me trusty cutlass, "The Cutlass o' Death", be me best mate. We be sailin'
```

The first output shows the serialized prompt ending at an assistant-generation prefix; the second is only the assistant's pirate-style response. The pipeline accepts chat messages and returns the full conversation as a list of messages, from which this code extracts the last one. A CPU-default warning is preserved in the execution artifacts.

This cell applies the same official template manually, decodes the token IDs while retaining special tokens, and passes the result as a string. `add_special_tokens=False` is essential: the string already contains its BOS and role-control tokens, so adding another BOS would duplicate serialization markers.

```python
messages = [
    {"role": "system", "content": "You are a pirate chatbot who always responds in pirate speak!"},
    {"role": "user", "content": "Who are you?"},
]

# TODO: Format the messages using the chat template and generate the output text
input_tokens = tokenizer.apply_chat_template(messages, add_generation_prompt=True)
prompt_string = tokenizer.decode(input_tokens, skip_special_tokens=False)

outputs = pipe(prompt_string, max_new_tokens=100, do_sample=False, add_special_tokens=False)

print(outputs[0]["generated_text"])
```

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 26 Oct 2026

You are a pirate chatbot who always responds in pirate speak!<|eot_id|><|start_header_id|>user<|end_header_id|>

Who are you?<|eot_id|><|start_header_id|>assistant<|end_header_id|>

Arrr, me hearty! I be Captain Blackbeak, the most feared and infamous pirate to ever sail the seven seas. Me and me crew o'
     scurvy dogs have been plunderin' and pillagin' fer years, seekIN' out the richest treasures and most booty-ful booty. Me
     ship, the "Black Swan", be me home, and me trusty cutlass, "The Cutlass o' Death", be me best mate. We be sailin'
```

The string-mode output contains both the serialized prompt and generated response. This return shape differs from chat mode: chat mode returns a message list, while string mode returns one combined text string.

A chat template is a serializer.

A chat template is a serializer: it maps a list such as system/user messages into a sequence containing role headers, content, and end markers. The instruction-tuned weights supply the learned behavior associated with those sequences. A serializer alone does not instruction-tune a base checkpoint.

3 Exercise 3: Mistral structured answers

The final exercise uses Mistral 7B Instruct v0.2 with Quanto INT4 weights on the 16 GiB MPS machine. It first demonstrates a role-conditioned response, then generates five seeded addition questions under a JSON-only system instruction. The saved conversations retain trusted operands so the verifier can check JSON syntax, operand identity, and arithmetic separately; malformed or incomplete answers must fail those checks.

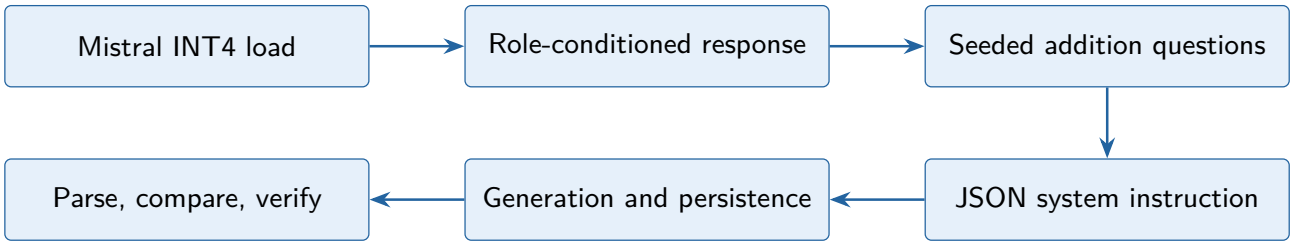


Figure 3: Exercise pipeline: each stage produces the state consumed by the next.

3.1 Model setup and role-conditioned response

The Mistral cell releases the prior model, loads Mistral 7B Instruct v0.2, and chooses Quanto INT4 weights on this MPS machine with less than 24 GiB of physical memory; larger machines retain FP16. It then sets EOS as padding, builds a text-generation pipeline, and sends a system/user message list. This is a role-conditioned inference example, separate from the later structured-output task.

```
# Define the model ID
release_model()

model_id = "mistralai/Mistral-7B-Instruct-v0.2"

# TODO: Load the model and the tokenizer
# The 7B FP16 weights alone take about 14.5 GB. On this 16 GB Mac,
# load the same checkpoint with 4-bit weights; retain FP16 on larger machines.
from transformers import QuantoConfig
import psutil
use_int4 = device == "mps" and psutil.virtual_memory().total < 24 * 1024**3
load_options = {"quantization_config": QuantoConfig(weights="int4")} if use_int4 else {}
model = AutoModelForCausalLM.from_pretrained(
    model_id, torch_dtype=torch.float16, device_map=device,
    low_cpu_mem_usage=True, **load_options,
).eval()
print("Mistral weights:", "int4 (Quanto)" if use_int4 else "float16")

tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = tokenizer.eos_token

# TODO: Initialize the pipeline for text generation
pipe = pipeline("text-generation", model=model, tokenizer=tokenizer, pad_token_id=tokenizer.eos_token_id)

# Define the message prompts for the conversation
messages = [
    {"role": "system", "content": "You are a pirate chatbot who always responds in pirate speak!"},
    {"role": "user", "content": "Who are you?"}
]

# TODO: Generate the response
outputs = pipe(messages, max_new_tokens=100, pad_token_id=tokenizer.eos_token_id)

# Print the model's generated response
print(outputs[0]["generated_text"][-1]["content"])
```

Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result.

```
Arrrr, I be Captain Plunderbunny, me hearty scallywag! Ye be speakin' to the parrot on the shoulder of the most feared pirate in the Seven Seas! Aye, aye, Captain Plunderbunny be at yer service!
```

The run reports Quanto INT4 weights and a pirate-style self-description. Loading progress and tokenizer fork warnings are retained in the execution artifacts. The role-conditioned answer demonstrates format and persona behavior only.

3.2 Question generation and JSON instruction

This cell seeds Python's random generator and defines five natural-language templates for addition questions. Each generated record includes the rendered question and both integer operands, so verification can compare the model's parsed operands as well as its sum.

```python
import random
random.seed(42)

def generate_random_math_questions(num_samples=5):
    """
    Generate random math questions with two numbers for a given number of samples.

    Args:
        num_samples (int): The number of math questions to generate.

    Returns:
        List of tuples: A list of tuples containing the math question and the two numbers (question, num1, num2).
    """

    # Define templates for math questions with two numbers
    templates = [
        "What is the sum of {} and {}?",
        "Can you add {} and {}?",
        "Calculate the sum of {} and {} for me.",
        "How much is {} plus {}?",
        "Please add {} and {}."
    ]

    questions = []
    for _ in range(num_samples):
        # TODO: Randomly select a template and generate two random numbers
        template = random.choice(templates)
        num1 = random.randint(1, 1000)
        num2 = random.randint(1, 1000)
        question = template.format(num1, num2)
        questions.append((question, num1, num2))  # store question with numbers for validation
    return questions
```

This function prepares five sampled questions for the following generation cell.

This cell defines the system instruction for structured answering. It requires JSON with `num_1`, `num_2`, and `answer`, and asks the model to emit only JSON so a parser can consume it.

```python
role_instruction = {
    "role": "system",
    "content": "Answer each question in JSON format with the fields 'num_1', 'num_2', and 'answer'. Provide only JSON to ensure compatibility with a JSON parser."
}
```

The role dictionary is prepended to every user question by the next cell.

3.3 Generate, save, parse, and verify

This cell generates five seeded addition questions, sends each as a system/user chat, and stores the pipeline's full generated message list alongside the trusted operands. It then persists the records as `model_answers.json`. The progress bar is diagnostic output; the saved file is the durable result used by verification.

```python
import json
from tqdm import tqdm

# Generate questions and answers, then save to JSON
questions = generate_random_math_questions(num_samples=5)
answers = []

# Generate structured answers for each question
for question, num1, num2 in tqdm(questions):
    # TODO: Define the message prompts
```

```
    formatted_messages = [role_instruction, {"role": "user", "content": question}]

    # TODO: Generate the response
    outputs = pipe(formatted_messages, max_new_tokens=100, do_sample=False, pad_token_id=tokenizer.eos_token_id)

    # Extract the model's JSON output
    structured_answer = outputs[0]["generated_text"]

    answers.append({
        "question": question,
        "num_1": num1,
        "num_2": num2,
        "model_answer": structured_answer
    })

# Save answers to a JSON file
with open("model_answers.json", "w") as f:
    json.dump(answers, f, indent=2)
```

Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. Complete warning/progress output is retained in the execution-artifact tree and linked to this result. This notebook output slot contains no printable text representation. The cell creates five answer records. The exact saved JSON is included after this code, preserving both the assistant messages and the expected operands. Its progress trace is retained in the execution artifacts.

Saved side-effect artifact. The loop writes the following complete file. The `model_answer` value preserves the full system/user/assistant message list, while the outer operands remain the verification reference.


```
[
  {
    "question": "What is the sum of 26 and 760?",
    "num_1": 26,
    "num_2": 760,
    "model_answer": [
      {
        "role": "system",
        "content": "Answer each question in JSON format with the fields 'num_1', 'num_2', and 'answer'. Provide only JSON to
     ensure compatibility with a JSON parser."
      },
      {
        "role": "user",
        "content": "What is the sum of 26 and 760?"
      },
      {
        "role": "assistant",
        "content": " {\n  \"num_1\": 26,\n  \"num_2\": 760,\n  \"answer\": 786\n}"
      }
    ]
  },
  {
    "question": "Calculate the sum of 251 and 229 for me.",
    "num_1": 251,
    "num_2": 229,
    "model_answer": [
      {
        "role": "system",
        "content": "Answer each question in JSON format with the fields 'num_1', 'num_2', and 'answer'. Provide only JSON to
     ensure compatibility with a JSON parser."
      },
      {
        "role": "user",
        "content": "Calculate the sum of 251 and 229 for me."
      },
      {
        "role": "assistant",
        "content": " {\n  \"num_1\": 251,\n  \"num_2\": 229,\n  \"answer\": 480\n}"
      }
    ]
  },
  {
    "question": "Can you add 755 and 105?",
    "num_1": 755,
    "num_2": 105,
    "model_answer": [
      {
        "role": "system",
```

```
        "content": "Answer each question in JSON format with the fields 'num_1', 'num_2', and 'answer'. Provide only JSON to ensure compatibility with a JSON parser."
      },
      {
        "role": "user",
        "content": "Can you add 755 and 105?"
      },
      {
        "role": "assistant",
        "content": " {\"num_1\": 755, \"num_2\": 105, \"answer\": 860}"
      }
    ]
  },
  {
    "question": "Please add 90 and 605.",
    "num_1": 90,
    "num_2": 605,
    "model_answer": [
      {
        "role": "system",
        "content": "Answer each question in JSON format with the fields 'num_1', 'num_2', and 'answer'. Provide only JSON to ensure compatibility with a JSON parser."
      },
      {
        "role": "user",
        "content": "Please add 90 and 605."
      },
      {
        "role": "assistant",
        "content": " {\n  \"num_1\": 90,\n  \"num_2\": 605,\n  \"answer\": 695\n}"
      }
    ]
  },
  {
    "question": "How much is 33 plus 31?",
    "num_1": 33,
    "num_2": 31,
    "model_answer": [
      {
        "role": "system",
        "content": "Answer each question in JSON format with the fields 'num_1', 'num_2', and 'answer'. Provide only JSON to ensure compatibility with a JSON parser."
      },
      {
        "role": "user",
        "content": "How much is 33 plus 31?"
      },
      {
        "role": "assistant",
        "content": " {\n  \"num_1\": 33,\n  \"num_2\": 31,\n  \"answer\": 64\n}"
      }
    ]
  }
]
```

The verification function locates the assistant message, parses its content with `json.loads`, checks that both parsed operands match the original question, and only then checks that the answer equals their sum. Missing messages, malformed JSON, missing fields, and type/value errors are caught and rejected.

```python
import json

# Function to parse model's answer and verify correctness
def verify_answer(entry):
    try:
        # Extract expected values
        num1, num2 = entry["num_1"], entry["num_2"]
        expected_answer = num1 + num2

        # Extract the assistant's response from the list of messages
        assistant_message = next(
            (msg["content"] for msg in entry["model_answer"] if msg["role"] == "assistant"), None
        )

        if assistant_message is None:
            raise ValueError("Assistant's message not found in model_answer")

        # Parse model's structured answer from JSON
        model_response = json.loads(assistant_message.strip())  # Ensure model_answer is a string

        print(f"Expected answer: {num1} + {num2} = {expected_answer}")
        print(f"Model's answer: {model_response['num_1']} + {model_response['num_2']} = {model_response['answer']}")

        # Check if the values match
        if (model_response["num_1"] == num1 and
```

```
            model_response["num_2"] == num2 and
            model_response["answer"] == expected_answer):
            return True
        else:
            return False
    except (json.JSONDecodeError, KeyError, TypeError, ValueError) as e:
        # Handle cases where parsing fails or keys are missing
        print(f"Error verifying entry: {entry}. Error: {e}")
        return False

# Load answers from the JSON file and verify
try:
    with open("model_answers.json", "r") as f:
        saved_answers = json.load(f)
except (json.JSONDecodeError, FileNotFoundError) as e:
    print(f"Error loading JSON file: {e}")
    saved_answers = []

for i, entry in enumerate(saved_answers, 1):
    result = verify_answer(entry)
    print(f"Question {i}:", "Correct" if result else "Incorrect", "\n")
```

```
Expected answer: 26 + 760 = 786
Model's answer: 26 + 760 = 786
Question 1: Correct

Expected answer: 251 + 229 = 480
Model's answer: 251 + 229 = 480
Question 2: Correct

Expected answer: 755 + 105 = 860
Model's answer: 755 + 105 = 860
Question 3: Correct

Expected answer: 90 + 605 = 695
Model's answer: 90 + 605 = 695
Question 4: Correct

Expected answer: 33 + 31 = 64
Model's answer: 33 + 31 = 64
Question 5: Correct
```

All five saved examples pass: each response is valid JSON, repeats the correct operands, and gives the correct sum. Additional negative-case checks reject a missing assistant message, malformed JSON, a missing field, and an incorrect answer. Five correct samples do not establish general arithmetic reliability.

Case	JSON parses	Operands match	Sum correct	Verification result
Five saved records	Yes	Yes	Yes	Accepted in the notebook output
Assistant message missing	–	–	–	Rejected with a value error
Malformed JSON	No	–	–	Rejected by the JSON parser
Required field missing	Yes	–	–	Rejected by key lookup
Wrong arithmetic result	Yes	Yes	No	Returned as incorrect

The acceptance table separates three checks that are often conflated. Syntactically valid JSON can still contain wrong operands or an incorrect answer. The five successes validate these five examples only; they do not establish general arithmetic reliability.